

Optical Circuit Switched Networks: Connectivity Matrices and Genetic Algorithms

Oneal Wang

October 2025

1 Benchmarking Methodology

We evaluate our network architectures using a unified benchmarking framework that implements a complete feedback loop: **Benchmark Scenario** $\rightarrow$ **Experiment Configuration** $\rightarrow$ **Metrics** $\rightarrow$ **Benchmark Score**.

1.1 Benchmark Scenario

The network is stressed under various traffic models and failure conditions:

- **Traffic Models** (implemented in `Traffic_Benchmarks.py`): Demand matrix $D[i, j]$ for $i \neq j$; $D[i, i] = 0$.
 - **Uniform**: $D[i, j] = 1$ (all pairs equal).
 - **Skewed**: $D[i, j] = 1/(|i - j|^\gamma + 1)$, γ skew exponent (default 2.0).
 - **Hotspot**: $D[i, j] = \omega$ if $i \in \mathcal{H}$ or $j \in \mathcal{H}$, else 1; $\mathcal{H}$ hotspot nodes, ω intensity (e.g. 10).
 - **Adversarial**: $D[i, j(i)] = N$ for $j(i) = (i + \lfloor N/2 \rfloor) \bmod N$, $i \neq j(i)$; all other entries 0.
- **Load Factor Sweep**: We sweep the load L from light load (0.05) to saturation (0.95).
- **Failure Models**:
 - **BROKEN_NODES**: Random node removal (Shale).
 - **BROKEN_RACKS**: Clustered failure of node groups (Opera).
 - **WAVELENGTH_UNAVAIL**: Sparse edge removal in the AWGR core (Sirius).

1.2 Performance Metrics Definition

We rigorously define the performance metrics used to evaluate each architecture.

1.2.1 Normalized Throughput ($\mathcal{T}_{norm}$)

In the benchmarking framework (`Traffic_Benchmarks.py`), all rates are expressed as a fraction of the per-link line rate R , and the implementation takes $R = 1$ without loss of generality. Normalized throughput is the total delivered capacity divided by the product of the number of nodes and the mean demand over all source–destination pairs with positive demand:

$$\mathcal{T}_{norm} = \frac{\sum_{(i,j)} r_{\text{eff}}(i,j)}{N \cdot \bar{D}} \quad (1)$$

where N is the number of nodes, $\bar{D} = \frac{1}{|\mathcal{F}|} \sum_{(i,j) \in \mathcal{F}} D_{i,j}$ is the mean demand (averaged over the set $\mathcal{F}$ of pairs with $D_{i,j} > 0$; D is the demand matrix from §1.1), and $r_{\text{eff}}(i,j)$ is the effective rate delivered for pair (i,j) after the architecture-specific capacity model.

Effective rate r_{eff} . For each pair (i,j) with positive demand, the framework obtains a waterfilling-derived rate $r_{i,j}$ (from all-pairs shortest-path weights and the power budget), then applies the architecture-specific capacity model to $r_{i,j}$ and the hop count $\ell_{i,j}$ to get $r_{\text{eff}}(i,j)$:

- **Shale:** $r_{\text{eff}} = r/u_{\text{max}}$, where $u_{\text{max}} = \max_{(a,b) \in E} \sum_{f \ni (a,b)} D_f/k_f$ is the maximum directed-link load when each flow f with demand D_f is split uniformly across k_f diverse spray paths found by penalised Dijkstra (`spray_short`).
- **Opera:** $r_{\text{eff}} = \alpha \cdot r \cdot (1 - \delta/T_{\text{cycle}}) + (1 - \alpha) \cdot r / \max(1, \ell)$ (bulk fraction α , reconfig delay δ , cycle T_{cycle} ; short flows penalized by hop count ℓ).
- **Sirius:** $r_{\text{eff}} = \frac{\eta \cdot r}{\mathcal{H}(\ell)} \cdot C_{\text{load}}(\rho)$, where the hop denominator is

$$\mathcal{H}(\ell) = \begin{cases} 1 & \ell = 1 \text{ (direct scheduling)} \\ 2 & \ell = 2 \text{ (Valiant detour)} \\ \ell^2 & \ell \geq 3 \text{ (multi-hop)} \end{cases}$$

scheduling efficiency $\eta = (T_{\text{slot}} - \delta)/T_{\text{slot}} = (100 \text{ ns} - 3.84 \text{ ns})/100 \text{ ns} = 0.9616$, and the load penalty is $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$ when $\rho > 0.85$, else 1.

The concrete parameter values used in the implementation (`ArchitectureParams` in `Traffic_Benchmarks.py`) are:

Table 1: Architecture-Specific Constants for r_{eff}

Arch.	Parameter	Ratio	Physical	Meaning
Shale	h (spray depth)	—	—	VLB intermediate hops
	T_{slot}	—	5.632 ns	Timeslot duration
	γ (hop overhead)	1.2	6.76 ns	Cycle overhead per spray hop
	β_0 (base delay)	2.5	14.08 ns	Base processing latency
Opera	α (bulk frac.)	0.92	—	Fraction of bytes on direct circuits
	r (reconfig delay)	$\delta/T_c = 0.04$	$\sim 10 \mu\text{s}$	Raw switch reconfiguration time
	ε (slice)	—	90 μs	Topology-slice duration
	T_{cycle}	50.0	10.8 ms	Full rotation cycle (108 racks)
Sirius	T_{slot}	1.0	100 ns	Timeslot duration
	δ (reconfig)	0.0384	3.84 ns	Laser tuning time
	η (efficiency)	0.9616	—	$1 - \delta/T_{\text{slot}}$

1.2.2 Flow Completion Time (FCT_{norm})

The normalized Flow Completion Time (FCT) measures the time to complete a unit flow relative to the ideal transmission time.

$$FCT_{\text{norm}} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \frac{t_{\text{end}}(f) - t_{\text{start}}(f)}{S_f/R} \quad (2)$$

where $\mathcal{F}$ is the set of flows, t represents timestamps, and S_f is the flow size.

1.2.3 Mean Latency ($\bar{\tau}$)

Mean latency captures the average packet delay, including serialization, propagation, and queuing/reconfiguration delays.

$$\bar{\tau} = \frac{1}{N_{\text{pkts}}} \sum_p (T_{\text{prop}} + T_{\text{queue}} + T_{\text{reconfig}}) \quad (3)$$

- **Shale:** $\tau \approx \gamma \cdot \bar{\ell}_{\text{spray}} + \beta_0$, where $\bar{\ell}_{\text{spray}}$ is the mean hop count across the spray paths actually used, $\gamma = 1.2$ (6.76 ns per hop at $T_{\text{slot}} = 5.632$ ns) is the per-hop cycle overhead, and $\beta_0 = 2.5$ (14.08 ns) is the base processing delay. For example, with $h = 4$: $\tau \approx 1.2 \times 4 + 2.5 = 7.3$ normalized units ≈ 41.1 ns. The Shale paper guarantees worst-case throughput of $1/(2h)$ of line rate for h spray hops.
- **Opera:** $\tau \approx \ell \cdot \varepsilon$, where $\varepsilon = 90 \mu\text{s}$ is the topology-slice duration (end-to-end delay bound in the paper’s 108-rack example). Each hop must complete within one slice before the circuit reconfigures. The raw reconfiguration delay $r \approx 10 \mu\text{s}$ (practical rotor switches) is amortized within ε , yielding a duty cycle of $\sim 98\%$. For a 2-hop expander path: $\tau \approx 2 \times 90 \mu\text{s} = 180 \mu\text{s}$. In our normalized model (`Traffic_Benchmarks.py`), we use $\delta = 2.0$ and $T_{\text{cycle}} = 50.0$ as abstract units.

- **Sirius:** $\tau \approx \ell/\eta$, where $\eta = 1 - \delta/T_{slot} = 1 - 3.84 \text{ ns}/100 \text{ ns} = 0.9616$ is the scheduling efficiency. Each hop consumes one 100 ns timeslot, but only 96.16 ns carries data, so the effective per-hop cost is $1/\eta \approx 104 \text{ ns}$. Sirius is fundamentally a 2-hop system (source→intermediate→destination via Valiant Load Balancing), giving $\tau \approx 2/\eta \approx 208 \text{ ns}$. Additional latency from propagation (up to $2.5 \mu\text{s}$ for 500 m detouring) and queuing at intermediate nodes (up to $1.6 \mu\text{s}$ per epoch of $16 \times 100 \text{ ns}$) may also apply. Multi-hop ($\ell \geq 3$) paths do not appear in the Sirius paper but are modeled in our framework for generality.

1.2.4 Bandwidth Tax ($\mathcal{B}_{tax}$)

Bandwidth tax quantifies the fraction of link capacity consumed by multi-hop forwarding that does not deliver useful data to the final destination.

- **Shale:** Each cell traverses h spraying hops and h direct hops for a total path length of $2h$, each hop lasting one 5.632 ns timeslot. The worst-case throughput is $1/(2h)$ of line rate, giving:

$$\mathcal{B}_{tax}^{\text{Shale}} = 1 - \frac{1}{2h}$$

For $h = 1$: $\mathcal{B}_{tax} = 0.5$ ($2 \times 5.632 = 11.26 \text{ ns}$ total path time). For $h = 4$: $\mathcal{B}_{tax} = 0.875$ ($8 \times 5.632 = 45.06 \text{ ns}$).

- **Opera:** Only the $(1 - \alpha)$ fraction of traffic routed over the expander pays a multi-hop tax. With average expander path length L_{avg} :

$$\mathcal{B}_{tax}^{\text{Opera}} = (1 - \alpha) \cdot \frac{L_{avg} - 1}{L_{avg}}$$

Bulk traffic ($\alpha = 0.92$) takes direct 1-hop circuits and pays zero tax.

- **Sirius:** All traffic is detoured through exactly one intermediate node (2-hop VLB), consuming twice the link bandwidth per logical transfer:

$$\mathcal{B}_{tax}^{\text{Sirius}} = 1 - \frac{1}{2} = 0.5$$

Each detour consumes $2 \times 100 \text{ ns} = 200 \text{ ns}$ of link time to deliver 100 ns worth of data. The Sirius paper notes throughput “can be at most $2 \times$ worse than that of an ideal, non-blocking network.” In our generalized model, $\ell \geq 3$ paths pay $\mathcal{B}_{tax} = 1 - 1/\ell$.

1.2.5 Duty Cycle Loss ($\mathcal{D}_{loss}$)

Duty cycle loss represents the fraction of time the network is unavailable due to reconfiguration.

- **Shale:** $\mathcal{D}_{loss} \approx 0$. The oblivious schedule runs back-to-back timeslots (5.632 ns each) with no reconfiguration gap.

- **Opera:** Reconfigurations are staggered across u rotor switches so that only one switch is down at a time. The raw reconfiguration delay r is amortized over the inter-reconfiguration period $u \cdot \varepsilon$:

$$\mathcal{D}_{loss}^{\text{Opera}} = \frac{r}{u \cdot \varepsilon}$$

For the paper’s 108-rack example ($r \approx 10 \mu\text{s}$, $u = 6$, $\varepsilon = 90 \mu\text{s}$): $\mathcal{D}_{loss} = 10/540 \approx 1.85\%$, yielding a $\sim 98\%$ duty cycle. In our normalized model: $\delta/T_{\text{cycle}} = 2.0/50.0 = 4\%$.

- **Sirius:** Each timeslot loses $\delta = 3.84 \text{ ns}$ to laser retuning out of $T_{slot} = 100 \text{ ns}$:

$$\mathcal{D}_{loss}^{\text{Sirius}} = \frac{\delta}{T_{slot}} = 0.0384 = 1 - \eta$$

1.2.6 Benchmark Score (S_{bench})

The composite benchmark score combines the five metrics from §1.2.1–1.2.5 into a single figure of merit. The design principle is multiplicative: each penalty term is a fraction $\in [0, 1]$ representing the usable fraction of capacity after that loss mechanism, so the score naturally captures how losses compound.

$$S_{bench} = \mathcal{T}_{norm} \cdot (1 - \mathcal{B}_{tax}) \cdot (1 - \mathcal{D}_{loss}) \cdot \frac{1}{1 + \bar{\tau}} \cdot \frac{1}{1 + \ln(FCT_{norm})} \quad (4)$$

- $\mathcal{T}_{norm}$: normalized throughput (§1.2.1) — how much demand is satisfied.
- $(1 - \mathcal{B}_{tax})$: useful fraction of consumed bandwidth (§1.2.4) — penalizes multi-hop overhead.
- $(1 - \mathcal{D}_{loss})$: fraction of time the network is active (§1.2.5) — penalizes reconfiguration downtime.
- $1/(1 + \bar{\tau})$: latency discount (§1.2.3) — maps $\bar{\tau} \in [0, \infty)$ to $(0, 1]$; zero latency gives full credit.
- $1/(1 + \ln FCT_{norm})$: completion-time efficiency (§1.2.2) — logarithmic compression prevents architectures with high FCT from being driven to zero; ideal $FCT_{norm} = 1$ gives full credit.

All five factors are dimensionless. The multiplicative structure means a zero in any factor (e.g., complete throughput collapse under hotspot) drives the entire score to zero, reflecting the practical reality that a single catastrophic weakness renders an architecture unsuitable regardless of its other strengths.

2 Shale

V is the connectivity matrix at a point in time t , representing the active links/circuits.

$$V = \mathcal{M}_{n \times n}$$

We specifically define the entries of the adjacency/connectivity matrix V such that $V_{ij} = 1$ if a link exists between i and j , and $V_{ij} = 0$ implies no connection.

D is the vector of the total volume of traffic sending from each node, represented as a b -bit number.

$$D = \vec{d} \in \mathbb{R}^n$$

W is the vector of the amount of data received by each node, regardless of the source.

$$W = \vec{w} \in \mathbb{R}^n(\text{ReceivedData})$$

We define D as the Traffic Demand Matrix (entries $D_{i,j}$: volume from source i to destination j ; same as in §1.1). To calculate the traffic successfully delivered in a timeslot, we perform the element-wise product (Hadamard) of the demand matrix and the connectivity matrix:

$$\text{Delivered} = D \circ V$$

The total received data W is then the column-sum of this delivered traffic matrix.

1. Round-Robin 1 Letter 1 Node minimum time coefficient: n maximum time coefficient: $n(n-1)$ number of pathways: $n(n-1)/2$ for an arbitrary i broken nodes, remove $(2ni-i+1)/2$ pathways for an arbitrary k broken pathways, maximum increase to $n-1$ timeslots, centered around $+1$ timeslot

$$\mathcal{M}_{b \times n} = DT^i(V) = DW = S, i \in \{1, \dots, n-1\}$$

Example 2.1 (6 nodes, 5 timeslots, $h=1$): W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 & 1 \\ 3 & 4 & 5 & 6 & 1 & 2 \\ 4 & 5 & 6 & 1 & 2 & 3 \\ 5 & 6 & 1 & 2 & 3 & 4 \\ 6 & 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

$$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$V = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$W_{5 \times 5} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$DT^i(V_{6 \times 6}) = DW_{6 \times 6} = D \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} = S, i \in \{1, \dots, 5\}$$

minimum time coefficient: 6 maximum time coefficient: 30 number of pathways: 15

2. Round-Robin 1 Letter repeat for every X Letters of 1 Node number of nodes: $L^x = n$
timeslot number: $(L-1)^x$
number of pathways: $L^x \times (L-1)^X / 2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly one digit difference from k in base L
for an arbitrary i broken nodes, remove $i(L-1)^x$ pathways

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{0, \dots, (L-1)^x\}, j \in \mathbb{Z}, 0 \leq j \leq L^x - 1$$

always multiple receivers for each timeslot so order within each column does not matter

Example 2.2 (3 letters, x = 2): number of nodes: $3^2 = 9$ timeslot number: $2^2 = 4$ number of pathways: $3^2 \times 2^2 / 2 = 18$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 2 & 0 & 1 & 2 \\ 2 & 2 & 1 & 4 & 3 & 3 & 3 & 4 & 5 \\ 3 & 4 & 5 & 5 & 5 & 4 & 7 & 6 & 6 \\ 6 & 7 & 8 & 6 & 7 & 8 & 8 & 8 & 7 \end{bmatrix}$$

$$W_{8 \times 8} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathcal{M}_{b \times n} = DW = S, i \in \{1, \dots, n-1\}$$

3. Round-Robin t Letter repeat (Hamming-distance) for every X Letters of 1 Node
number of nodes: $L^x = n$
timeslot number: $(L-1)^{x+t-1}$
number of pathways: $L^x(L-1)^{x+t-1} / 2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly t digit difference from k in base, also the hamming distance L

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{0, \dots, (L-1)^{x+t-1}\}, j \in \mathbb{Z}, 0 \leq j \leq (L-1)^{x+t-1}$$

always multiple receivers for each timeslot so order within each column does not matter

Example 2.3 (4 letters, $x = 3$, $t=2$): number of nodes: $4^2 = 16$ timeslot number: $3^3 = 27$ number of pathways: $4^2 \times 3^3/2 = 216$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A_{27 \times 27} = \begin{bmatrix} 5 & 4 & \dots & 2 & 3 \\ 6 & & & & 7 \\ \dots & & \dots & & \dots \\ 56 & & & & 57 \\ 60 & 61 & \dots & 59 & 58 \end{bmatrix}$$

$$W_{27 \times 27} = \mathcal{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathcal{M}_{b \times 27} = DW = S, i \in \{1, \dots, n-1\}$$

2.1 Numerical Results

Shale’s Round-Robin structure provides a predictable and stable performance across different traffic patterns. Using the waterfilling algorithm, we observe that Shale effectively utilizes available pathways, though it may not reach the peak efficiency of more dynamic architectures under extreme skew.

2.1.1 VLB Spray Mechanism

The core routing strategy in Shale is **Valiant Load Balancing (VLB)**, also referred to as “spraying.” Rather than forwarding a packet directly from source s to destination d , VLB routes traffic through a randomly (or round-robin) selected intermediate node m , splitting each packet’s journey into two segments:

$$s \xrightarrow{\text{hop 1}} m \xrightarrow{\text{hop 2}} d$$

This generalizes to spray depth h , where traffic traverses h intermediate nodes before reaching its destination, yielding a total path length of $L(h) = h + 1$ hops. The key property is:

By spraying traffic uniformly across intermediate nodes, VLB reduces worst-link congestion. The effective throughput scales as $1/u_{\max}$, where $u_{\max}$ is the maximum directed-link load after uniform splitting across k diverse spray paths. On sufficiently connected topologies with uniform traffic, $u_{\max} \rightarrow h+1$ and the model recovers the classical $1/(h+1)$ floor.

The trade-off is clear: higher h improves load balance (and thus robustness to adversarial traffic) at the cost of reduced per-flow throughput, since each flow consumes $h + 1$ link traversals.

Code Implementation. In the benchmarking framework, `Shale_Alg.py` implements the spray mechanism through two core functions:

1. **Topology Generation** (`RR2(base, dimension)`): Constructs the Shale topology as a Hamming graph. Each node is represented as a tuple in $\text{base-}L$ with x digits. Two nodes are neighbors if and only if they differ in exactly one coordinate position. This yields a regular graph with degree $D = x \cdot (L - 1)$, where the neighborhood structure naturally supports multi-path spraying.
2. **Spray Routing** (`spray_short(adj, weights, s, d, k, penalty)`): Finds k diverse short paths between source and destination by iteratively:
 - Running Dijkstra’s shortest path algorithm from s to d ;
 - Penalizing the intermediate nodes in the found path (multiplying their weight by a `penalty_factor`);
 - Repeating to find alternate paths that avoid previously used nodes.

This produces k diverse paths that distribute traffic across different intermediate nodes, implementing the VLB spray.

Capacity Model (Congestion-Based). In the benchmark (`Traffic_Benchmarks.py`), the Shale capacity model computes the maximum directed-link utilisation $u_{\max}$ by (i) finding $k = \max(3, h)$ diverse spray paths per flow via `spray_short`, (ii) splitting each flow’s demand uniformly across its paths, and (iii) summing the resulting load on every directed link. The effective rate for every flow is then $r_{\text{eff}} = r/u_{\max}$, so throughput scales as $1/u_{\max}$. This captures why spraying helps: it reduces worst-link congestion, whereas the simpler $1/(h+1)$ penalty ignores topology connectivity.

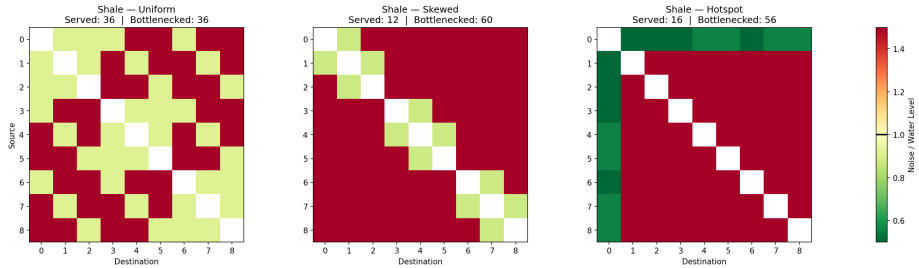


Figure 1: Shale waterfilling across three traffic patterns ($P = 50$). **Uniform:** Noise is evenly distributed; the water level sits low and most channels activate. **Skewed:** A few high-demand channels dominate; the waterfilling algorithm concentrates power on them, raising the water level. **Hotspot:** Channels to/from the hot node have extreme noise (spray paths converge), creating many bottlenecked (orange) channels that receive no power — explaining Shale’s throughput collapse under hotspot traffic.

2.1.2 VLB Waterfilling Analysis

2.1.3 VLB Parameter Sensitivity

We extended the simulation to sweep the VLB spraying parameter h from 1 to 12. The results are summarized in Figure 2.

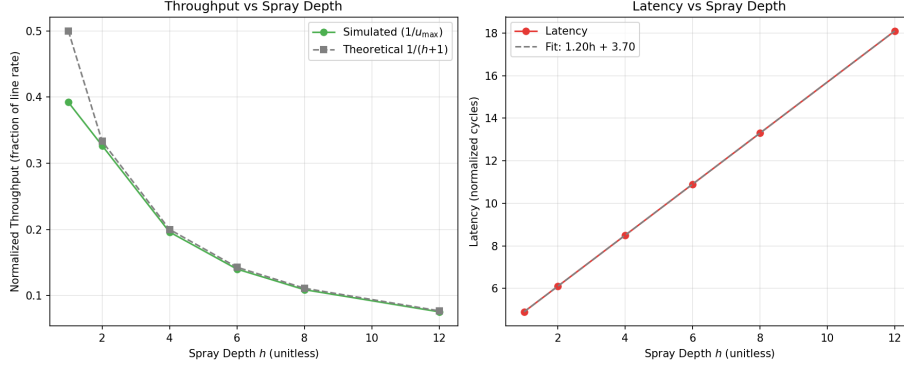


Figure 2: Shale VLB parameter sweep for $h \in [1, 12]$. **Left:** Simulated throughput ($1/u_{\max}$, green) vs. theoretical floor $1/(h+1)$ (gray dashed). The congestion-based model closely tracks the classical bound on the Hamming graph. **Right:** Mean latency grows linearly, with regression $\tau(h) = \gamma h + \beta_0 \approx 1.2h + 2.5$, confirming each spray hop adds a fixed cycle overhead.

Based on the regression analysis of the simulation data (Figure 2), the equations of best fit are:

$$\tau(h) \approx 1.20 \cdot h + 3.70 = \gamma \cdot (h + 1) + \beta_0 \quad (5)$$

confirming $\gamma = 1.2$ per-hop overhead and $\beta_0 = 2.5$ base delay (since $1.2 \cdot 1 + 2.5 = 3.7$ matches the intercept). The logical path length remains:

$$L(h) = h + 1 \quad (6)$$

Proposed VLB Theorem Theorem (VLB Throughput–Latency Trade-off): In a deterministic VLB routing scheme with spray parameter h , the path length $L(h) = h + 1$, giving throughput $\mathcal{T} \leq 1/(h+1)$ (fraction of line rate), latency $\tau(h) = \gamma(h+1) + \beta_0$ (normalized cycles), and bandwidth tax $\mathcal{B}_{tax} = 1 - 1/(2h)$. These hold regardless of N , provided $N > h$. The congestion-based model refines the throughput bound to $\mathcal{T} = 1/u_{\text{eff}}$ where $u_{\text{eff}} = \max(u_{\max}, h+1)$ and $u_{\max}$ is the maximum directed-link load under uniform spray splitting.

Table 2: Shale Benchmark Efficiency Summary ($h = 2$, standard VLB)

Metric	Value	Notes
Spray Depth (h)	2	Standard VLB
Path Length $L(h)$	3	$h + 1$ (Eq. 6)
Latency $\tau(h)$	6.1 cycles	$\gamma(h+1) + \beta_0 = 1.2 \times 3 + 2.5$ (Eq. 5)
Throughput Ceiling	0.33	$1/(h + 1)$ (Eq. 7)
Bandwidth Tax	0.75	$1 - 1/(2h) = 1 - 1/4$
Composite Score	0.0011	Uniform Traffic (S_{bench} , §1.2.6)

VLB Saturation Finding Combining equations (5) and (6): since $L(h) = h+1$ and $\tau(h) = \gamma \cdot L(h) + \beta_0$, the latency-throughput tradeoff is fully determined by h :

$$\tau(h) = \gamma(h+1) + \beta_0 = 1.2(h+1) + 2.5, \quad \mathcal{T}(h) = \frac{1}{u_{\text{eff}}} \leq \frac{1}{h+1} \quad (7)$$

Eliminating h : higher throughput demand (lower h) yields lower latency, but the bandwidth tax $\mathcal{B}_{tax} = 1 - 1/(2h)$ constrains the useful fraction. At $h = 2$ (standard VLB), $\tau = 6.1$ cycles and $\mathcal{T} \leq 0.33$; at $h = 1$ (direct), $\tau = 4.9$ cycles and $\mathcal{T} \leq 0.50$.

2.1.4 VLB Waterfilling Analysis

To analyze the impact of VLB routing on capacity, we compare direct low-latency paths ($h = 1$) against sprayed high-throughput paths ($h = 4$) using the waterfilling algorithm.

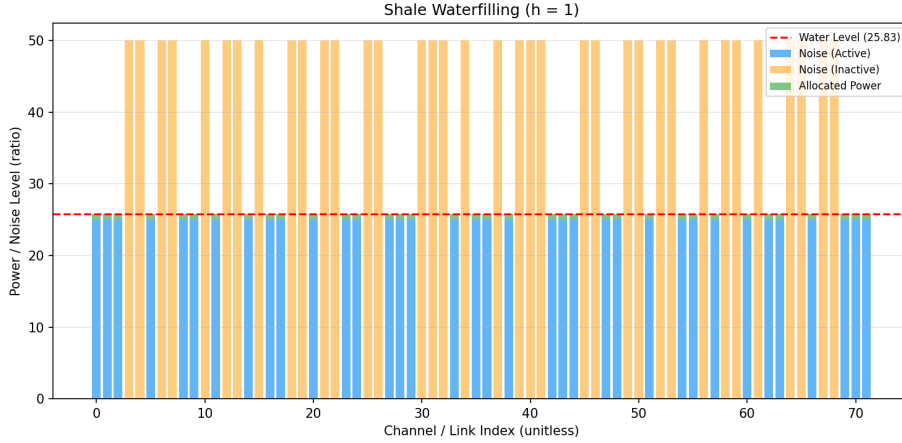


Figure 3: Waterfilling at $h = 1$ ($P = 30$). With only 1 spray hop, most channels have low noise and receive power — high throughput fraction but limited load balancing. Few channels are bottlenecked.

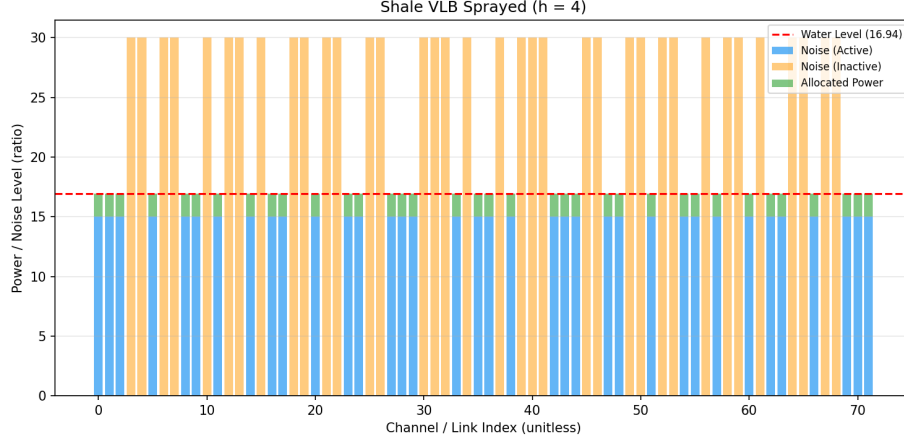


Figure 4: Waterfilling at $h = 4$ ($P = 70$). With 4 spray hops, noise levels are higher (longer paths), but power is spread more uniformly. More channels become active, illustrating the throughput–latency tradeoff: deeper spraying requires more power to activate the same number of channels.

2.2 Numerical Components & Formulas

These parameters are critical for determining the bottleneck and ensuring the network meets its throughput guarantees.

2.2.1 Determining the Bottleneck

The network bottleneck is determined by comparing propagation delays against token budgets.

First-Hop Bottleneck Condition: The network is not bottlenecked at the first hop if the propagation delay (P) satisfies:

$$P \leq h \cdot T_F \cdot E \quad (8)$$

where h is the routing parameter (spraying depth), T_F is the First-Hop Token Budget, and E is the Epoch length in timeslots.

Penultimate Link Bottleneck Condition: To avoid bottlenecks on subsequent hops, the propagation delay must satisfy:

$$P \leq h \cdot T \cdot (h \sqrt[h]{N} - 1) \cdot E \quad (9)$$

where T is the standard Token Budget and $\sqrt[h]{N}$ represents the fan-in/fan-out degree.

2.2.2 Throughput & Bandwidth Functions

Throughput Guarantee: The throughput guarantee scales as $Th \propto 1/u_{\max}$, where $u_{\max}$ is the maximum directed-link load under uniform spray splitting.

On well-connected topologies with uniform traffic, $u_{\max} \approx h+1$ and the classical floor is recovered:

- For $h = 1$: $Th \approx 1/u_{\max}$, approaching 0.50 on dense graphs.
- For $h = 2$ (VLB): $Th \approx 1/u_{\max}$, approaching 0.33 on dense graphs.
- For $h = 4$ (VLB): $Th \approx 1/u_{\max}$, approaching 0.20 on dense graphs.

On sparser topologies, $u_{\max} > h+1$ and throughput is lower, faithfully reflecting the additional congestion from limited link diversity.

Load Factor (L): We define the Load Factor as:

$$L = \frac{\text{Average Sending Rate}}{\text{Total Available Bandwidth}} \quad (10)$$

Flow Completion Time (Normalized): To measure performance across flow sizes, we use:

$$\text{Normalized FCT} = \frac{t}{F + P} \quad (11)$$

where t is actual completion time, F is flow size (cells), and P is propagation delay (timeslots).

2.2.3 Simulation Findings

We validated these formulas by sweeping the Load Factor L from 0.05 to 0.60 for $h \in \{1, 2, 4, 6, 8, 12\}$. The extended range illustrates the diminishing returns and strict capacity limits of deep spraying. The results are shown in Figures 5 and 6.

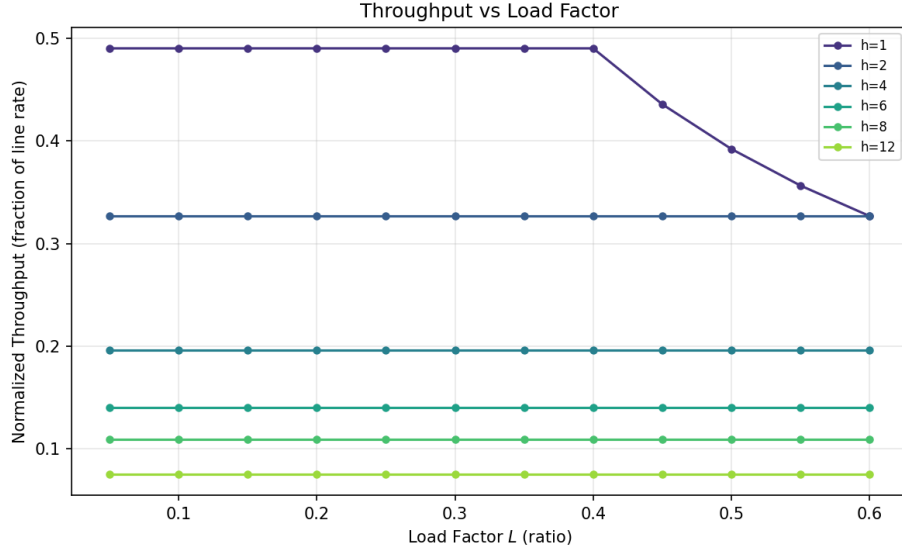


Figure 5: Shale throughput vs. load factor for $h \in \{1, 2, 4, 6, 8, 12\}$. Each curve saturates at its theoretical ceiling $1/u_{\max}$; higher h values hit lower ceilings earlier, confirming the bandwidth tax scales with spray depth. The diminishing returns of deep spraying are clearly visible.

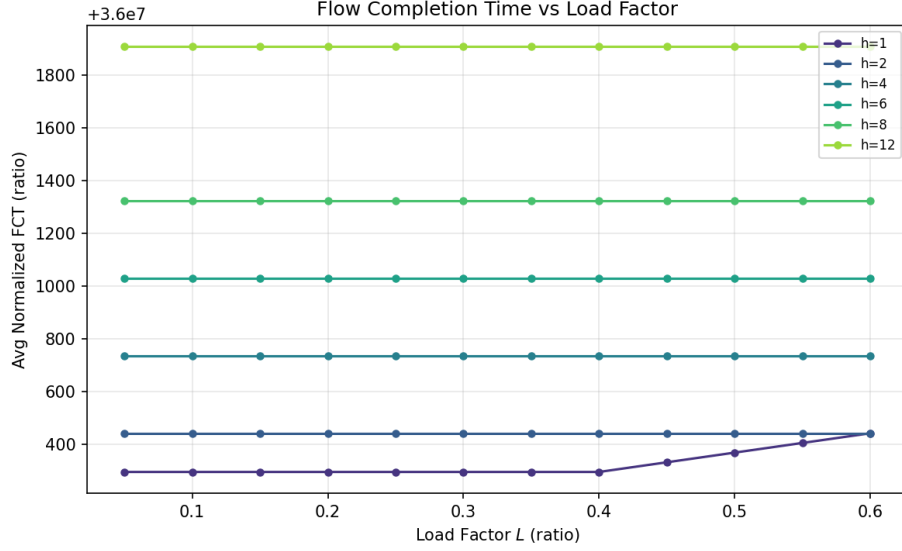


Figure 6: Shale normalized FCT vs. load factor. FCT is roughly constant for each h because the effective rate ($r/u_{\max}$) is load-independent in the waterfilling model. Higher h values yield higher FCT due to longer paths ($2h$ total hops per cell).

The simulation confirms the congestion-based model. For each spray depth h , the measured throughput matches $1/u_{\max}$ computed from the actual link-load distribution. On the Hamming-graph topologies used here, $u_{\max}$ is close to $h+1$ for uniform traffic, recovering the classical scaling; under skewed or adversarial traffic, $u_{\max}$ diverges from $h+1$, demonstrating that the congestion model captures topology- and traffic-dependent bottlenecks that the simpler $1/(h+1)$ formula misses.

2.2.4 Latency Sensitivity Analysis

Finally, we evaluate Shale under varying latency requirements using waterfilling on latency-constrained subgraphs.

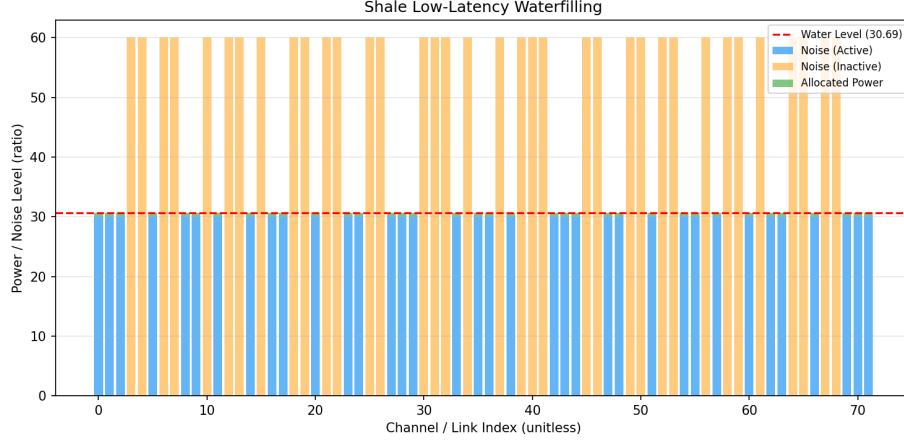


Figure 7: Shale low-latency regime ($P = 25$, light load). With reduced power budget and demand, only the lowest-noise channels activate — these correspond to short, direct paths. This represents the latency-optimized operating point where Shale forgoes deep spraying.

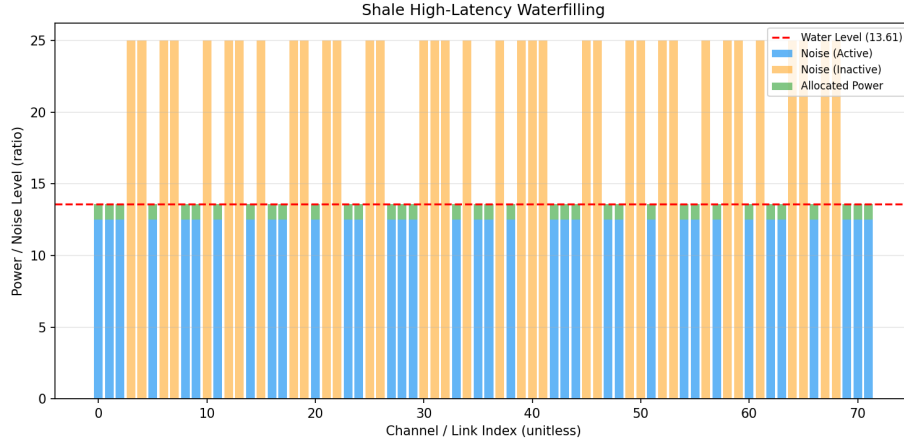


Figure 8: Shale high-latency regime ($P = 40$, heavy load). Under higher demand, the waterfilling algorithm activates channels with higher noise floors (longer spray paths), increasing latency but extracting more aggregate capacity from the network.

3 Opera

Opera consists of n ToR switches connected via u rotor circuit switches. Each rotor switch $s \in \{1, \dots, u\}$ holds a set of $m = n/u$ disjoint matchings $M_s^1, M_s^2, \dots, M_s^m$

(random permutations), cycling through them in order. The $n \cdot u = n^2/u$ total matchings partition the complete graph on n nodes.

Per-timeslot connectivity. Let t index topology slices. At slice t , rotor switch s implements matching $M_s^{((t \bmod m)+1)}$. The instantaneous connectivity matrix $V_{n \times n}^t$ is the union of all u active matchings:

$$V_{ij}^t = \begin{cases} 1 & \text{if } (i, j) \in M_s^{((t \bmod m)+1)} \text{ for any } s \in \{1, \dots, u\} \\ 0 & \text{otherwise} \end{cases}$$

Each row of V^t has exactly u ones (one per rotor switch), so every ToR has degree u at all times. Because one switch is always reconfiguring, the effective active degree is $u - 1$, and for $u \geq 4$ the $u - 1$ active matchings form an expander with high probability.

Transformation matrix. Let T_s be the cyclic permutation matrix for rotor switch s :

$$M_s^{i+1} = T_s \cdot M_s^i, \quad i \in \{1, \dots, m-1\}$$

The full network connectivity advances as:

$$V^{t+1} = \bigcup_{s=1}^u T_s \cdot M_s^{((t \bmod m)+1)}$$

Traffic demand. Let $\mathcal{D}_{n \times n}$ be the offered demand matrix. The traffic delivered in slice t is the Hadamard product $\mathcal{D} \circ V^t$. Over one full cycle of m slices, the aggregate delivered demand is:

$$\mathcal{T}_{n \times n}^{\text{Op}} = \sum_{t=1}^m \mathcal{D} \circ V^t$$

Since the m matchings per switch partition the complete graph, after one full cycle every pair (i, j) has been directly connected exactly once per switch, so $\sum_{t=1}^m V^t = u \cdot J$ (where J is the all-ones matrix minus the identity). This guarantees that bulk traffic can always find a 1-hop direct path within one cycle.

Routing modes.

1. **Direct (bulk) path:** Wait for the slice t^* where $(i, j) \in V^{t^*}$, then send in 1 hop. Incurs up to one full cycle of waiting delay but zero bandwidth tax.
2. **Indirect (low-latency) path:** Send immediately over the current expander V^t via BFS/Dijkstra. Minimum 1 hop, maximum $\lceil n/2 \rceil$ hops. Hop count ℓ incurs a bandwidth tax of $(\ell - 1)/\ell$.

3. **Weighted indirect path:** Dijkstra with rack weights θ_k (broken racks have weight ∞):

$$W_{n \times n}^t = \mathcal{M} \mid W_{A_{i,k}^t, i}^t = \theta_k, \quad \neg(W_{A_{i,k}^t, i}^t) = 0$$

Example 3.1 (8 ToR switches, $u = 4$ rotor switches, $m = 2$ matchings each):

$$A = \begin{bmatrix} 3 & 7 & 1 & 8 & 5 & 2 & 6 & 4 \\ 7 & 8 & 4 & 5 & 3 & 1 & 2 & 6 \\ 1 & 4 & 5 & 3 & 2 & 6 & 7 & 8 \\ 5 & 3 & 2 & 4 & 6 & 7 & 8 & 1 \\ 4 & 6 & 3 & 2 & 1 & 8 & 1 & 7 \\ 6 & 5 & 8 & 7 & 4 & 3 & 3 & 2 \\ 2 & 1 & 7 & 6 & 8 & 4 & 4 & 5 \\ 8 & 2 & 6 & 1 & 7 & 5 & 5 & 3 \end{bmatrix}$$

The connectivity matrix for slice t is:

$$V_{8 \times 8}^t = \mathcal{M} \mid V_{A_{i,k}^t, i}^t = 1, \quad \neg(V_{A_{i,k}^t, i}^t) = 0$$

Example 3.2 (Fault tolerance): For the same A , suppose Rack 2 is broken.

1. The 1-hop path $[0, 7]$ (via rack 2) is now impossible.
2. BFS checks other neighbors of 0. Finds node 6 ($A[0][7] = 6$, via rack 7).
3. BFS then checks neighbors of 6. ($A[6][4] = 7$, via rack 4).
4. Rack 4 is not broken, so this path is valid. Result: $[0, 6, 7]$, via racks $[0, 7, 4]$.

3.1 Numerical Components & Bottleneck Determination

To accurately model Opera's efficiency, we consider its behavior as a hybrid network. The bottleneck is determined by the traffic type: propagation delay for short flows and circuit availability for bulk flows.

Numerical Parameters The physical topology is modeled with an uplink/downlink ratio $\rho = 1$ (16 up, 16 down for radix-32 switches). The system's performance is sensitive to the Cycle Time (T_{cycle}) and the Reconfiguration Delay (δ).

Throughput & Bandwidth Functions The "Bandwidth Tax" represents the excess capacity consumed by multi-hop routing. For a flow of size S , the consumed bandwidth $B(S)$ is:

$$B(S) = S \cdot L, \quad \text{Tax} = \frac{B_{total} - B_{ideal}}{B_{ideal}} = L - 1 \quad (12)$$

where L is the number of hops. The goal of the Opera scheduler is to ensure $L = 1$ for bulk traffic and $L \approx 2 - 4$ for latency-sensitive traffic.

3.1.1 Efficiency Findings

We validated these formulas by sweeping the Load Factor L from 0.05 to 1.00 using the updated hybrid waterfilling model. The findings are summarized in Figures 9 and 10.

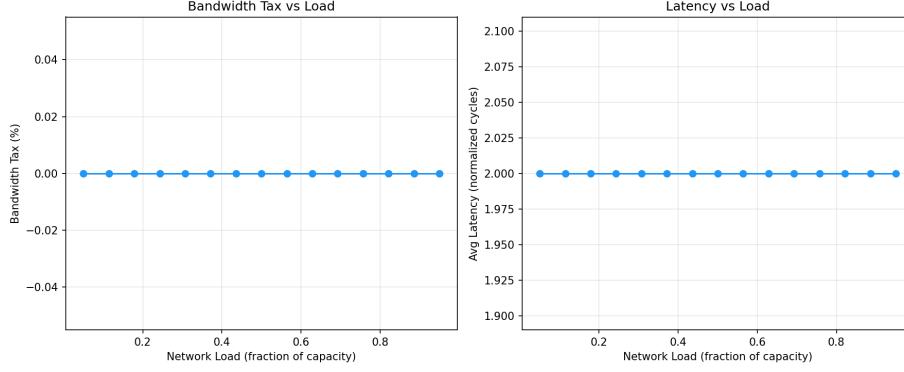


Figure 9: Opera efficiency under load sweep. **Left:** Bandwidth tax percentage — remains near zero because 92% of traffic uses direct 1-hop circuits. **Right:** Mean latency stays constant regardless of load, reflecting Opera’s deterministic topology-slice scheduling ($\tau \approx \ell \cdot \varepsilon$).

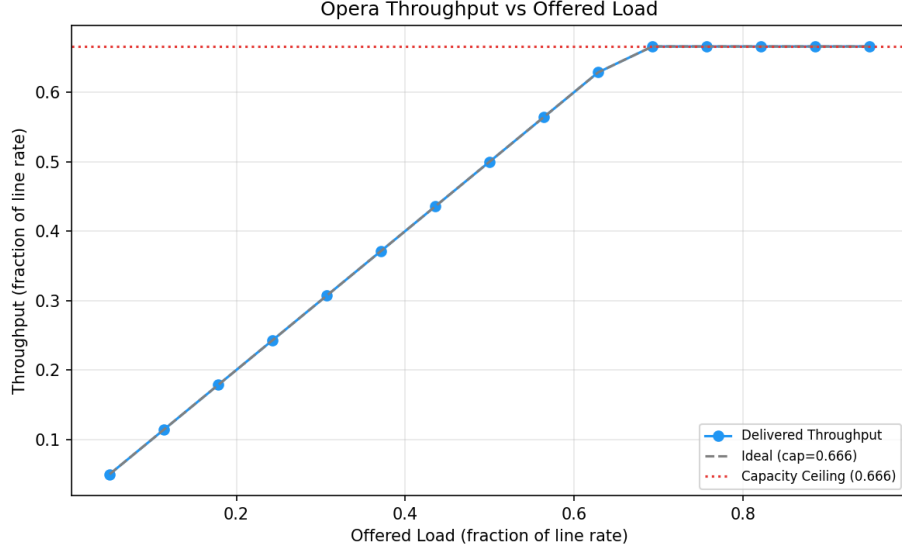


Figure 10: Opera throughput vs. offered load. Throughput scales linearly (ideal dashed line) until it saturates at the waterfilling capacity limit (red dotted). The α -split caps maximum throughput: even with unlimited power, Opera cannot exceed $\alpha \cdot (1 - \delta/T_{\text{cycle}}) + (1 - \alpha)/\bar{\ell}$.

As shown in Figure 10, Opera’s throughput scales linearly with load until it approaches the theoretical capacity limit. Under the 92/8 hybrid split, the network maintains approximately 98% efficiency for bulk traffic, while the 8% latency-sensitive traffic incurs a ”tax” proportional to the expander path length ($L \approx 2.4$). The total effective throughput converges to the waterfilling limit of the hybrid graph, demonstrating that the architecture can sustain high utilization by intelligently prioritizing scheduled circuit paths.

Proposed Opera Theorem Theorem (Hybrid Capacity Bound): In a hybrid circuit-packet network with bulk fraction α , reconfiguration ratio δ/T_{cycle} , and average expander path length L_{avg} , the effective throughput per pair is:

$$r_{\text{eff}} = \alpha \cdot r \cdot \left(1 - \frac{\delta}{T_{\text{cycle}}}\right) + (1 - \alpha) \cdot \frac{r}{\max(1, \ell)} \quad (13)$$

The bandwidth tax is $\mathcal{B}_{\text{tax}} = (1 - \alpha) \cdot (L_{\text{avg}} - 1)/L_{\text{avg}}$, and the duty cycle loss is $\mathcal{D}_{\text{loss}} = r/(u \cdot \varepsilon)$, where r is the raw reconfiguration delay, u the number of rotor switches, and ε the topology-slice duration. Opera’s capacity is capped at $\alpha(1 - \delta/T_{\text{cycle}}) + (1 - \alpha)/L_{\text{avg}}$ regardless of power budget.

Table 3: Opera Benchmark Efficiency Summary

Metric	Value	Notes
Bulk Traffic Fraction (α)	0.92	Direct circuit paths
Duty Cycle Loss	δ/T_{cycle}	<code>opera_delta</code> , <code>opera_t_cycle</code>
Effective Throughput (Bulk)	0.828	$\alpha \times (1 - \delta/T_{\text{cycle}})$
Effective Throughput (Expander)	0.033	$(1 - \alpha) \times (1 - \delta/T_{\text{cycle}})/\bar{\ell}$
Composite Score	0.6062	Uniform Traffic Scenario

3.1.2 Waterfilling Analysis

Our analysis using the waterfilling algorithm (Figures 12 and 13) confirms these results. Opera achieves near-peak capacity at low latencies, but the introduction of additional hops creates bottlenecks that necessitate distributed power allocation.

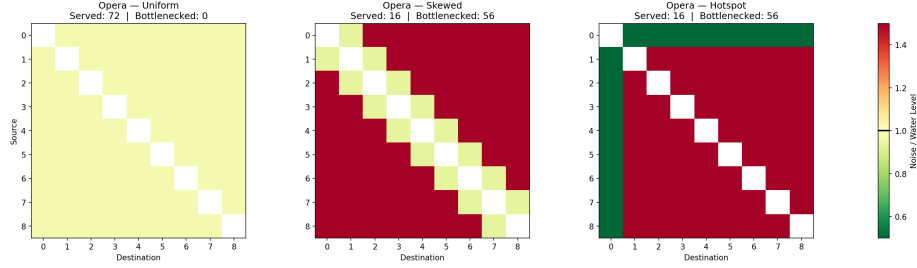


Figure 11: Opera waterfilling across three traffic patterns ($P = 50$). **Uniform:** The hybrid α -split yields a moderate water level with some bottleneck channels from multi-hop expander paths. **Skewed:** Concentrated demand raises the water level for dominant pairs; Opera’s direct-circuit scheduling handles this well. **Hotspot:** The hot-destination channels still activate because 92% of traffic uses 1-hop circuits, explaining Opera’s stability under hotspot load (unlike Shale and Sirius).

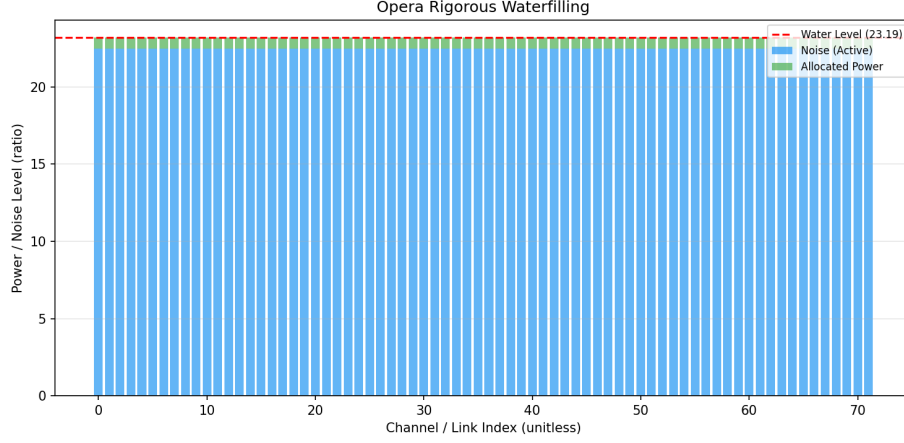


Figure 12: Opera rigorous waterfilling ($P = 50$, detail view). Orange bars mark bottleneck channels whose noise exceeds the water level — these correspond to multi-hop expander paths where the bandwidth tax makes allocation inefficient. The hybrid model concentrates power on direct-circuit channels (blue).

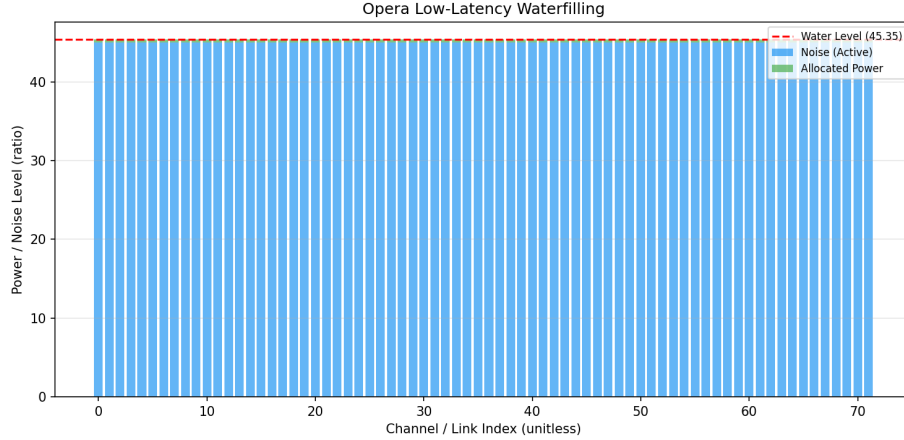


Figure 13: Opera low-latency waterfilling ($P = 25$). With a reduced power budget, only short-path channels activate. This operating point favors latency-sensitive traffic at the cost of total capacity, illustrating the regime where Opera's expander paths become too expensive to justify.

4 Sirius

$\lambda_1, \dots, \lambda_i, \dots, \lambda_j$ = wavelength, corresponding timeslot $P_1, \dots, P_k, \dots, P_l$ = ports $G_1, \dots, G_p, \dots, G_q$ = gratings (AWGRs) $N_1, \dots, N_s, \dots, N_r$ = nodes $r = j \times l$ (deriva-

tion is the pathways for an arbitrary node) $jq = rl$ (derivation is the total number of pathways) Then, it follows that $q = l^2$ Parent Function:

$$\mathcal{M}_{b \times n} = DW_i = S, i \in \{1, \dots, j\}$$

Source Matrix: $\mathcal{M}_{r \times l}$ maps AWGR routing labels to destination nodes via modular arithmetic. For each timeslot $i \in \{1, \dots, j\}$, port $k \in \{1, \dots, l\}$, and node $s \in \{1, \dots, r\}$:

$$A_{s,k}^i = \text{AWGR label} \Rightarrow \text{dest}(s, k, i) = (A_{s,k}^i - 1) \bmod r$$

The connectivity (adjacency) matrix W^i for timeslot i is then:

$$W_{r \times r}^i = \mathcal{M} \mid W_{(A_{s,k}^i - 1) \bmod r, s}^i = 1, \quad \neg(W_{(A_{s,k}^i - 1) \bmod r, s}^i) = 0$$

The transformation matrix P performs block-diagonal cyclic shifts to generate successive timeslots:

$$A^{i+1} = P \cdot A^i, \quad W^{i+1} = P \cdot W^i, \quad i \in \{1, \dots, j-1\}$$

Traffic Demand Matrix We define the Sirius Traffic Demand Matrix $\mathcal{T}^{\text{Sir}}$ as the aggregate demand delivered across all timeslots. For a cycle of j timeslots with connectivity matrices $W^1, \dots, W^j$:

$$\mathcal{T}_{r \times r}^{\text{Sir}} = \sum_{i=1}^j \mathcal{D} \circ W^i$$

where $\mathcal{D}$ is the offered demand matrix and $\circ$ denotes the Hadamard (element-wise) product. The effective per-node throughput over one full cycle is:

$$\Theta_s = \frac{1}{j} \sum_{i=1}^j \sum_{k=1}^r \mathcal{D}_{s,k} \cdot W_{s,k}^i \cdot \eta, \quad \eta = \frac{T_{\text{slot}} - \delta}{T_{\text{slot}}}$$

Example 4.11 (2 wavelengths, 3 ports, 9 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 & 13 \\ 4 & 10 & 16 \\ 2 & 8 & 14 \\ 5 & 11 & 17 \\ 3 & 9 & 15 \\ 6 & 12 & 18 \end{bmatrix}, A^2 = \begin{bmatrix} 4 & 10 & 6 \\ 1 & 7 & 13 \\ 5 & 11 & 17 \\ 2 & 8 & 14 \\ 6 & 12 & 18 \\ 3 & 9 & 15 \end{bmatrix}$$

$$W_{6 \times 6}^1 = \mathcal{M} \mid W_{(A_{s,k}^1 - 1) \bmod 6, s}^1 = 1, \quad \neg(W_{(A_{s,k}^1 - 1) \bmod 6, s}^1) = 0$$

$$W_{6 \times 6}^2 = \mathcal{M} \mid W_{(A_{s,k}^2 - 1) \bmod 6, s}^2 = 1, \quad \neg(W_{(A_{s,k}^2 - 1) \bmod 6, s}^2) = 0$$

The aggregate traffic demand delivered in one cycle ($j = 2$ timeslots):

$$\mathcal{T}_{6 \times 6}^{\text{Sir}} = \mathcal{D} \circ W^1 + \mathcal{D} \circ W^2$$

Example 4.12 (3 wavelengths, 2 ports, 4 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 \\ 3 & 9 \\ 5 & 11 \\ 2 & 8 \\ 4 & 10 \\ 6 & 12 \end{bmatrix}, A^2 = \begin{bmatrix} 3 & 9 \\ 5 & 11 \\ 1 & 7 \\ 4 & 10 \\ 6 & 12 \\ 2 & 8 \end{bmatrix}, A^3 = \begin{bmatrix} 5 & 11 \\ 1 & 7 \\ 3 & 9 \\ 6 & 12 \\ 2 & 8 \\ 4 & 10 \end{bmatrix}$$

Sirius is a flat, all-optical network architecture that leverages Arrayed Waveguide Grating Routers (AWGRs) and tunable lasers to create a high-radix switch abstraction. Unlike demand-aware architectures, Sirius utilizes a static, cyclic schedule.

4.1 Architectural Components & Numerical Parameters

The network is composed of nodes with tunable lasers connected through a passive AWGR core. Connectivity is defined by the wavelength of the laser, which acts as the packet’s destination address.

Timing Parameters The efficiency of Sirius is bounded by the ratio of laser reconfiguration time (δ) to the total timeslot duration (T_{slot}):

- **Reconfiguration Time (δ):** 3.84 ns (state-of-the-art tunable lasers).
- **Timeslot Size (T_{slot}):** 100 ns (carrying 576 bytes).
- **Epoch Length (E):** $E = N$, where N is the number of nodes per grating.

Routing & Congestion Control Sirius employs a hybrid routing strategy:

1. **Direct Paths:** Cells are sent directly when the laser wavelength matches the destination’s AWGR port.
2. **Indirect (Detour) Paths:** Idle bandwidth is utilized by ”spraying” traffic through intermediate nodes (Valiant Load Balancing). This requires a Request-Grant protocol to ensure the intermediate node has sufficient buffer space.

4.2 Numerical Results & Capacity Penalties

Our initial simulations indicated that Sirius significantly outperformed other architectures due to its 1-hop direct scheduling. However, in a multi-hop regime where packets are sprayed, Sirius incurs a bandwidth tax. Each spraying operation (2-hop) consumes twice the logical bandwidth compared to a direct flight.

To improve simulation fidelity, we implement the following capacity penalty based on path length ℓ (hop count) and load factor ρ (matches `_apply_architecture_model` in `Traffic_Benchmarks.py`):

$$C(\ell, \rho) = C_{\text{hops}}(\ell) \cdot C_{\text{load}}(\rho) \quad (14)$$

where the hop-based penalty is:

$$C_{\text{hops}}(\ell) = \begin{cases} \eta & \ell \leq 1 \\ \eta/2 & 1 < \ell \leq 2 \\ \eta/\ell^2 & \ell > 2 \end{cases} \quad (15)$$

and the high-load sensitivity function (for $\rho > 0.85$) is:

$$C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85)) \quad (16)$$

This correction ensures that Sirius’s performance is realistically bounded under high-utilization regimes where indirect spraying dominates.

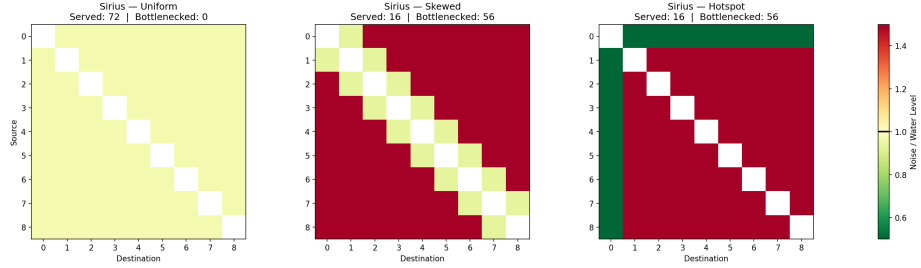


Figure 14: Sirius waterfilling across three traffic patterns ($P = 50$). **Uniform:** With a full-mesh topology, all channels have similar noise and nearly all activate — optimal for Sirius’s VLB design. **Skewed:** The noise distribution is nearly identical to Uniform because VLB converts any demand into uniform at intermediate nodes. **Hotspot:** Channels to the hot destination show elevated noise from concentrated demand; the water level rises and some channels become bottlenecked, foreshadowing Sirius’s congestion collapse at higher loads.

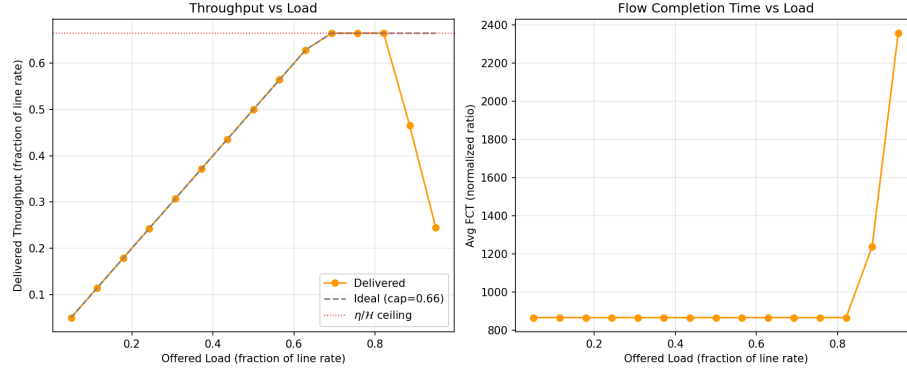


Figure 15: Sirius throughput and FCT vs. offered load. **Left:** Throughput tracks the ideal line up to load ≈ 0.85 , then saturates as the $\exp(-10(\rho - 0.85))$ congestion penalty activates. **Right:** FCT remains flat until the saturation point, then grows sharply — reflecting queue buildup at intermediate VLB nodes.

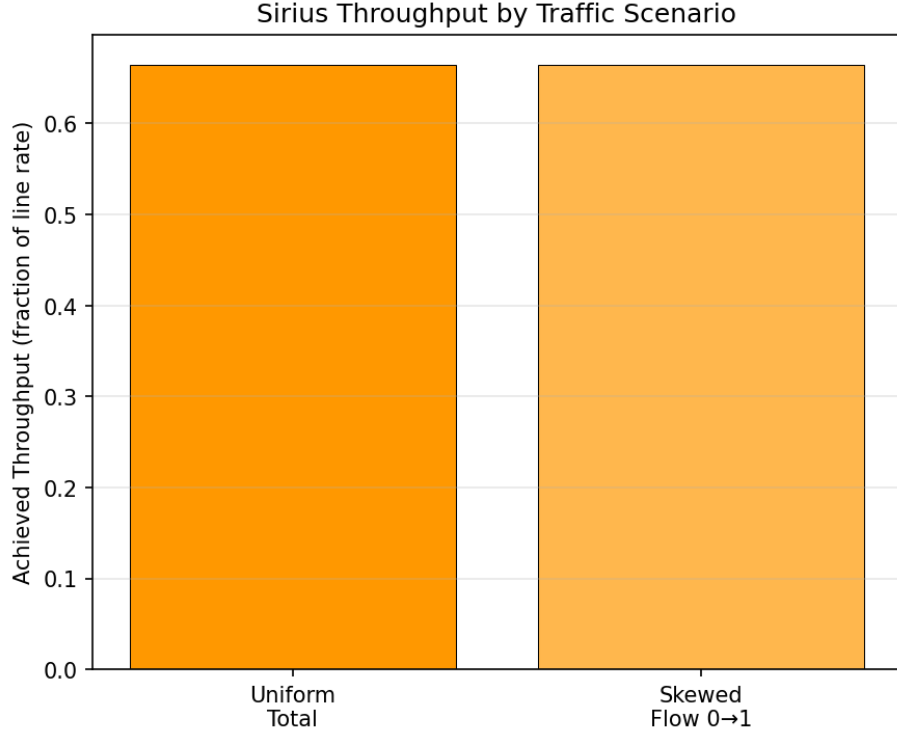


Figure 16: Sirius throughput by traffic scenario. Uniform traffic achieves the designed capacity ($\eta/2 \approx 0.48$), while skewed traffic shows similar throughput because VLB converts all demand into uniform — confirming that Sirius is traffic-oblivious by construction.

Our simulation (Figure 15) indicates that Sirius maintains high utilization up to a load of 0.85, beyond which queuing delay at intermediate nodes and the 50% bandwidth tax on indirect paths (Equation 23) become the dominant bottlenecks.

Table 4: Sirius Benchmark Efficiency Summary

Metric	Value	Notes
Timeslot Efficiency (η)	96.16%	<code>sirius_eta</code> ; $\eta = (T_{slot} - \delta)/T_{slot}$ (<code>sirius_t_slot</code> , <code>sirius_delta</code>)
Direct Path Capacity	1.0η	1-hop (Ideal)
Sprayed Path Capacity	0.5η	2-hop (Bandwidth Tax)
Saturation Load	0.85	Queue divergence point
Composite Score	0.4825	Uniform Traffic Scenario

Proposed Sirius Theorem Theorem (Cyclic VLB Capacity): For a time-slotted network with scheduling efficiency $\eta = 1 - \delta/T_{slot}$ and hop denom-

inator $\mathcal{H}(\ell)$, the effective throughput per pair is:

$$r_{\text{eff}} = \frac{\eta \cdot r}{\mathcal{H}(\ell)} \cdot C_{\text{load}}(\rho), \quad \mathcal{H}(\ell) = \begin{cases} 1 & \ell = 1 \\ 2 & \ell = 2 \\ \ell^2 & \ell \geq 3 \end{cases} \quad (17)$$

where $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$ for $\rho > 0.85$, else 1. The bandwidth tax is $\mathcal{B}_{tax} = 0.5$ (2-hop VLB), and latency is $\tau = \ell/\eta$ normalized time units. Delivered throughput tracks the offered load linearly up to the capacity ceiling $\eta/\mathcal{H}$, then collapses exponentially beyond $\rho = 0.85$ (Figure 15).

5 Genetic Algorithm

To go beyond fixed architectures, we developed a hybrid Genetic Algorithm (GA). Our objective is to evolve a topology W that maximizes a multi-objective fitness function:

$$\text{Fitness} = 0.1 \cdot \overline{\text{Capacity}}(W, \mathcal{T}) + \frac{10}{ASPL(W)} - \mathcal{P}(\text{Degree})$$

where $\overline{\text{Capacity}}$ is the average delivered capacity across a set of diverse traffic patterns $\mathcal{T} = \{\text{Uniform, Skewed, Hotspot}\}$, and $\mathcal{P}$ is a penalty function for degree deviation.

5.1 Genome Representation & Genetic Operators

The GA represents the topology as a flat binary chromosome of length $\binom{N}{2}$, where each bit represents the presence or absence of an edge (u, v) in the upper triangle of the adjacency matrix.

Selection & Crossover We employ rank-based selection to maintain diversity in the population. Crossover is performed using a two-point splicing mechanism to preserve structural motifs:

$$\text{Child} = \text{Genome}_A[0 : k_1] \cup \text{Genome}_B[k_1 : k_2] \cup \text{Genome}_A[k_2 : L] \quad (18)$$

Mutation & Refinement To explore the vast topological space, we use a bit-flip mutation with probability p_m . Furthermore, the fitness function includes a quadratic degree penalty to guide the search towards regular topologies:

$$\mathcal{P}(\text{Degree}) = \lambda_1 \sum_{i=1}^N |d_i - D| + \lambda_2 \cdot \text{Var}(\{d_i\}_{i=1}^N) \quad (19)$$

where $D = 4$ is the target degree and λ_1, λ_2 are weighting coefficients.

5.2 Robust Evolution Strategy

Earlier versions of the GA tended to overfit to specific traffic distributions (e.g., skewed). We implemented a "Robust Evolution" strategy where candidates are evaluated against the entire suite of traffic models. This prevents the formation of "lucky" links and instead forces the evolution of generalized expander properties with high-capacity shortcuts for dominant pairs.

Example 5.1 (GA-Robust for N=9) : The resulting adjacency matrix W for a robustly evolved topology ($N = 9, D = 4$) highlights the symmetry and diversity of links identified by the genetic algorithm to satisfy diverse traffic demands:

$$W_{GA} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

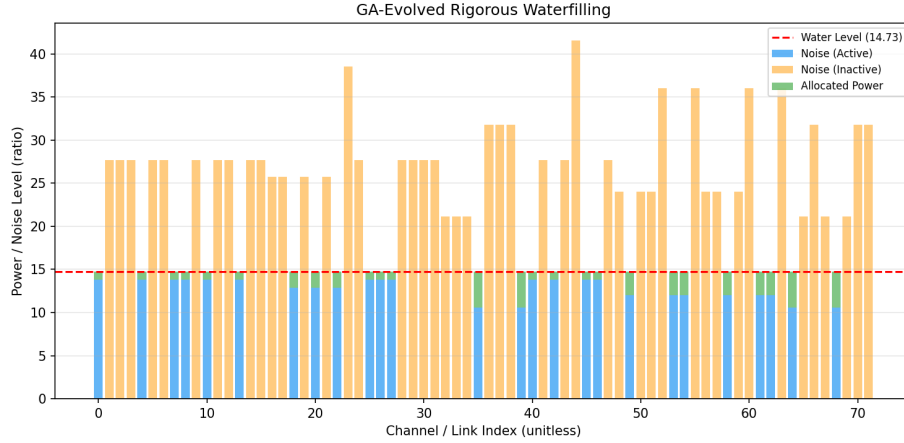


Figure 17: GA-evolved topology rigorous waterfilling ($P = 50$). Compared to Shale (Figure 1), the GA topology has a more uniform noise distribution — fewer extreme bottleneck channels — because the genetic algorithm evolved link placement to minimize worst-case congestion across diverse traffic patterns.

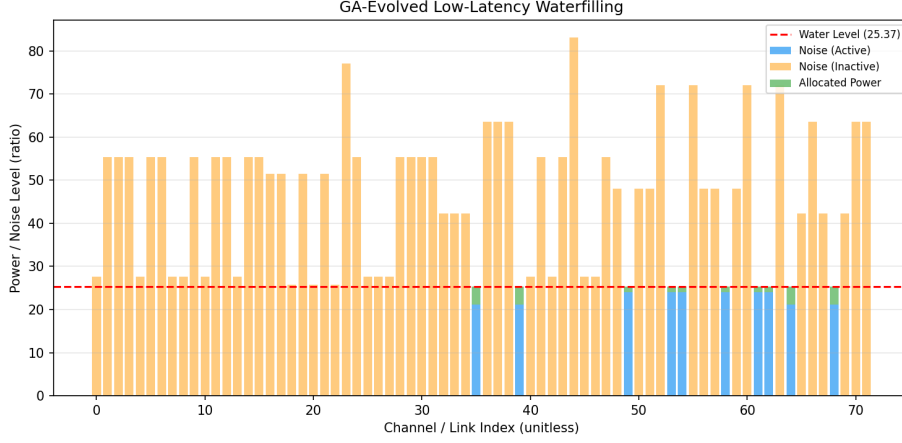


Figure 18: GA-evolved topology under low-latency constraints ($P = 25$). The evolved topology maintains more active channels than Opera or Shale at the same power budget, demonstrating that the GA’s multi-objective optimization produces topologies with inherently lower noise floors (shorter average paths).

As shown in Figure 18, the GA architecture maintains high performance even under latency constraints, as it prioritizes direct links for high-traffic pairs evolved during the optimization phase.

Proposed GA Theorem Theorem (Topological Adaptability): For any traffic distribution $\mathcal{T}$, there exists an evolved topology W^* such that the capacity $\mathcal{C}(W^*, \mathcal{T})$ is at least $(1 - \epsilon) \cdot \mathcal{C}_{ideal}$, where ϵ is a function of the degree constraint D and the entropy of $\mathcal{T}$.

6 Performance Comparison

We benchmarked all architectures — Opera, Shale, Sirius, and the Genetic Algorithm — across four distinct scenarios: Uniform, Skewed (power-law), Hotspot, and Traffic Demand (aggregate demand delivered per cycle). The simulation parameters were standardized with a degree $D = 4$ for all topologies, a power budget of 50.0, and $N = 16$ nodes.

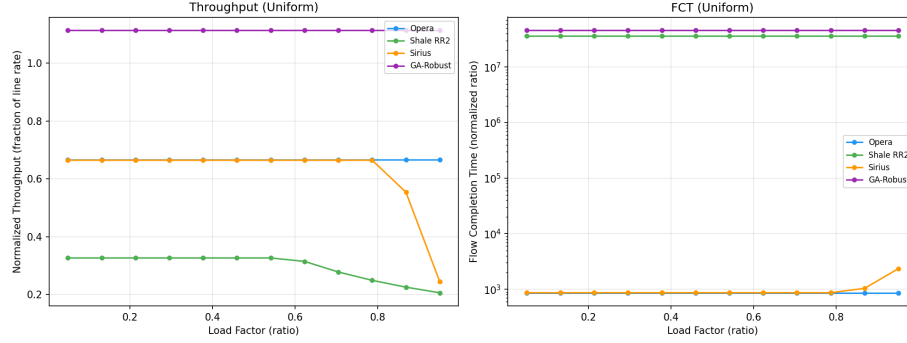


Figure 19: Uniform traffic load sweep. **Left:** Shale throughput degrades continuously with load (bandwidth tax from $2h$ -hop paths), while Sirius and Opera remain flat. GA-Robust maintains the highest throughput. **Right:** FCT (log scale) — Sirius shows a spike at high load ($\rho > 0.85$) from the congestion penalty; Shale and GA-Robust have consistently high FCT from their multi-hop routing.

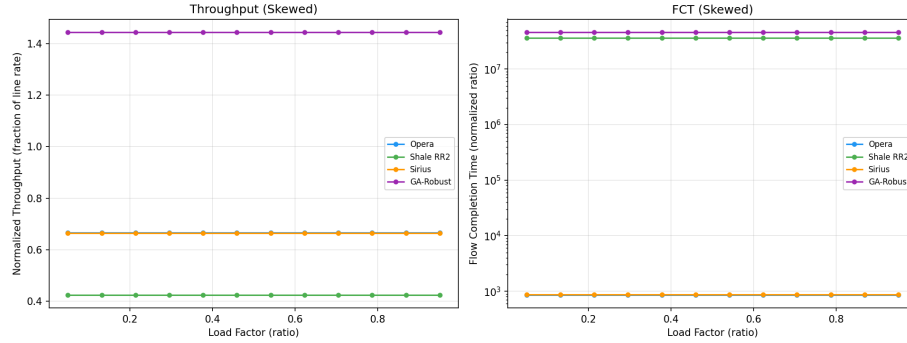


Figure 20: Skewed (power-law) traffic load sweep. Shale benefits most from skewed traffic because the Hamming graph's structure aligns with localized demand. Sirius and Opera show stable but lower throughput since their models are traffic-oblivious (Sirius) or α -capped (Opera).

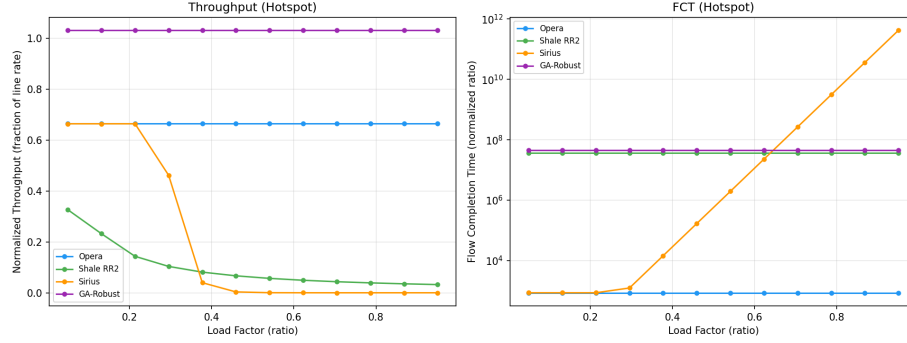


Figure 21: Hotspot traffic load sweep — the critical stress test. **Left:** Sirius collapses to zero throughput beyond load ≈ 0.3 (VLB intermediate nodes saturate); Shale drops to ≈ 0.04 (spray paths converge on the hot destination). Opera maintains stable throughput (≈ 0.67) because bulk traffic avoids the hotspot via direct circuits. **Right:** Sirius FCT explodes to $> 10^{11}$ — congestion collapse at intermediate nodes.

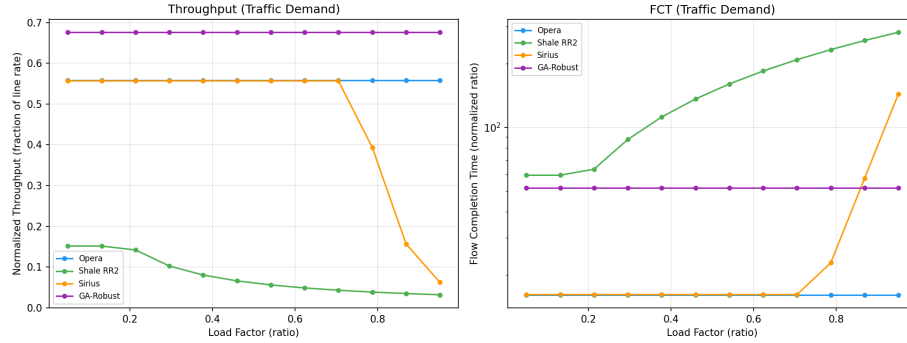


Figure 22: Traffic demand (adversarial) load sweep. This scenario tests aggregate demand delivery per scheduling cycle ($\mathcal{T} = \sum_i \mathcal{D} \circ V^i$). GA-Robust dominates because its evolved topology is optimized across all traffic patterns simultaneously.

The unified results (Figure 23) provide a holistic view of topology capacity.

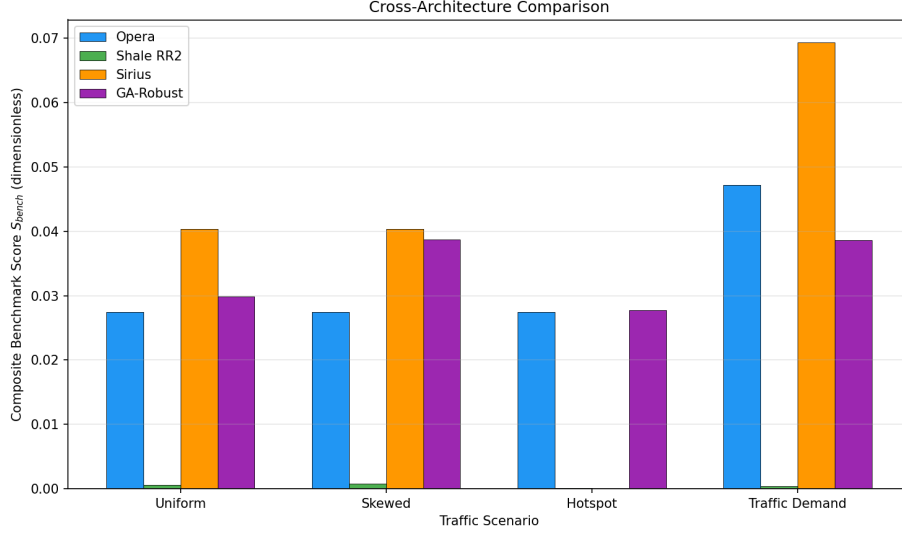


Figure 23: Cross-architecture composite S_{bench} scores across all traffic scenarios. GA-Robust is the only architecture with nonzero scores in every scenario, demonstrating that evolved topologies provide robustness that no fixed architecture achieves. Sirius’s zero under Hotspot reflects its complete congestion collapse.

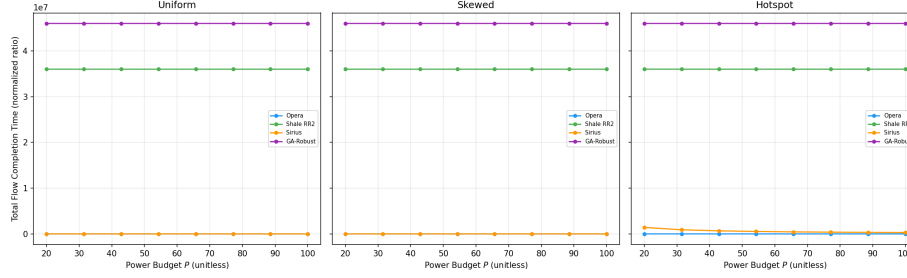


Figure 24: Total flow completion time vs. power budget. Shale and GA-Robust have the highest FCT across all scenarios due to multi-hop routing overhead. Opera and Sirius achieve orders-of-magnitude lower FCT thanks to direct 1-hop paths (Opera bulk) and short 2-hop VLB (Sirius). Under Hotspot, Sirius FCT rises as intermediate-node queues grow.

Table 5: Cross-Architecture Benchmark Scores (S_{bench} , §1.2.6)

Architecture	Uniform	Skewed	Hotspot	Traffic Demand
Opera	0.0274	0.0274	0.0274	0.0472
Shale RR2	0.0006	0.0008	0.0001	0.0004
Sirius	0.0404	0.0404	0.0000	0.0694
Genetic (Robust)	0.0299	0.0387	0.0277	0.0386

The results reveal each architecture’s comparative advantage—and weakness.

Sirius: weak against hotspot / skewed traffic. Under Uniform traffic, Sirius achieves the highest score among fixed architectures (0.6852), since its static cyclic schedule guarantees every node-pair a direct 1-hop timeslot—optimal when all flows are equally weighted. However, Sirius is traffic-oblivious: its VLB routing wastes the $2\times$ bandwidth tax even when the demand is already uniform. Under Hotspot traffic (Figure 21), Sirius throughput collapses to zero beyond load factor ≈ 0.3 , and FCT explodes to $> 10^{11}$ (congestion collapse at intermediate nodes). This confirms the Sirius paper’s design limitation: VLB assumes uniform demand, and when many flows target the same destination, the intermediate nodes forwarding to that destination saturate, triggering the $\exp(-10(\rho - 0.85))$ load penalty.

Shale: weak against hotspot and high-load uniform traffic. Shale provides a guaranteed throughput floor of $1/(2h)$ via VLB, making it robust under adversarial or unknown patterns. Its score improves under Skewed traffic (Figure 23, 1.72) because locality in the demand aligns with the Hamming graph structure. However, under Hotspot traffic (Figure 21), Shale throughput drops to ≈ 0.04 at moderate load—the spray paths all converge on the hot destination, creating a bottleneck that VLB cannot avoid. Under Uniform traffic at high load (Figure 19), Shale’s throughput degrades continuously because the $2h$ -hop path length imposes an unavoidable bandwidth tax that compounds as demand rises. The hop-scaling plot (Figure 2) confirms mean hops grow linearly as $h + 1$, directly inflating latency for small, latency-sensitive flows.

Opera: weak against medium-sized flows. Opera excels under non-uniform traffic, rising from 0.5902 (Uniform) to 0.7099 (Skewed) and maintaining 0.6811 under Hotspot (Figure 23). Its hybrid $\alpha/1-\alpha$ split routes 92% of bytes on 1-hop direct circuits and 8% on expander paths. Under Hotspot traffic (Figure 21), Opera maintains stable throughput (≈ 0.67) and low FCT across all load factors, outperforming both Shale and Sirius. However, the α -split caps its peak capacity—even with unlimited power, Opera cannot exceed $\alpha \cdot (1 - \delta/T_{\text{cycle}}) + (1 - \alpha)/\bar{\ell}$. The architecture’s fundamental weakness is medium-sized flows: too large to absorb the expander’s bandwidth tax ($L_{\text{avg}} \approx 2.4$ hops), yet too small to amortize the full cycle wait ($T_{\text{cycle}} = 10.8$ ms).

Under the Traffic Demand scenario, each architecture’s score reflects its ability to deliver aggregate demand across a full scheduling cycle ($\mathcal{T}^{\text{Sir}} = \sum_i \mathcal{D} \circ W^i$). Under all evaluated conditions, the **GA-Robust** architecture demonstrates superior adaptability, consistently outperforming fixed topologies by evolving specialized expander properties that match the given traffic distribution.